

Akshar Devanagari IME

Source Manual — Architecture, Mathematics, Training and Evaluation

Akshar IME

6 September 2026

Contents

1	About this manual	4
1.1	Conventions	4
1.2	Reproducing everything in this manual	4
2	What this system is	5
2.1	What it is not	5
2.2	Measured performance	5
3	Quick start	7
3.1	Build and install (Linux / IBus)	7
3.2	Use from Rust	7
3.3	Use from the browser	7
4	Problem formulation	8
4.1	The task	8
4.2	The source-channel decomposition	8
4.3	The unit of modelling: the akshara	9
4.4	Alignment is latent	9
5	The transliteration model and EM training	10
5.1	Parameters	10
5.2	Initialisation	10
5.3	The E-step: scaled forward-backward	10
5.3.1	Why the passes are scaled	11
5.4	The M-step	11
5.5	What the model looks like when trained	12
6	The language model	13
6.1	Why Kneser-Ney	13
6.2	Modified Kneser-Ney discounts	13
6.3	The interpolated model	14
6.4	Storage and lookup	14
6.5	Two known deviations from textbook KN	14
7	Decoding	15
7.1	The lattice	15
7.2	Beam search	15
7.3	Two passes: free and trie-constrained	16
7.4	What the decoder returns	16
8	Discriminative reranking	17

8.1	The baseline heuristic	17
8.2	Dense features	17
8.3	Sparse lexicalized features	18
8.4	The blend	18
8.5	The cascade	19
9	The engine: candidate fusion and learning	20
9.1	Sources of evidence	20
9.2	Digits and punctuation	20
9.3	Adaptive learning	21
9.3.1	How fuzzy matching actually behaves	21
10	The model container	22
10.1	One file	22
10.2	Versions	22
10.3	Compact encoding	22
10.4	Browser profile	23
11	Training	24
11.1	What --reranker-pairs does and does not control	24
11.2	The four phases	24
11.3	Learning rate	25
11.4	Held-out monitoring	25
11.5	Data	25
12	Evaluation	26
12.1	Harnesses	26
12.2	The accuracy regression guard	26
12.3	Headroom, and where the errors are	26
12.4	The collision bound	27
12.5	Statistical method	27
13	Ablations: what each component is worth	28
13.1	Switches	28
13.2	Contribution to native top-1	28
13.3	Paired significance	29
13.4	Components that were removed	29
13.5	Modified Kneser-Ney against a single discount	30
14	Performance	31
14.1	Where the time goes	31
14.2	How it got there	31
14.3	Trade-offs available	32
15	Deployment	33
15.1	Linux / IBus	33
15.2	Browser / WebAssembly	33
15.3	Offline tools	33
16	Source map	35
17	Known defects and limitations	37
17.1	Open defects	37

17.2 Things that do not work as their names suggest	37
17.3 Scope limitations	37
18 Roadmap	39
18.1 Result: more supervision was tested and does not help	39
18.2 The likely cause, and the corrected next step	40
18.3 What would falsify the remaining plan	40
19 References	41
20 Glossary	42

Chapter 1

About this manual

This is the complete reference for Akshar Devanagari IME: what it computes, how it is trained, how it is measured, and how to reproduce every number in it. It is written to be read start to finish by someone who has never seen the codebase.

Every quantitative claim here was measured on the tree it describes. Where a figure has not been re-measured since a change, the text says so rather than carrying an older number forward. Where a component does not work, the manual says that too — Chapter 11 is a register of known defects, and the ablations in Chapter 9 include the components that turned out to contribute nothing.

1.1 Conventions

- Weights are **negative log probabilities** throughout. Lower is better, and costs add. This is the tropical semiring, which is why decoding is a shortest-path problem.
- R denotes a Roman-script input string, D a Devanagari output string, a an *akshara* (orthographic syllable), s a Roman *chunk*.
- Accuracy figures are **top-1 exact string match** unless stated otherwise.
- AK-Freq, AK-NEF, AK-NEI are the three sources of the Aksharantar Nepali test split: frequent native words, and two named-entity sets.
- Code references are given as `path/to/file.rs`.

1.2 Reproducing everything in this manual

```
make release      # build the engine
make test         # unit + integration tests, including the accuracy guard
make eval         # accuracy by split
make eval-full    # accuracy with bootstrap CIs and per-query latency
make eval-errors  # oracle curves, error taxonomy, CER, collision bound
```

Component ablations are switched at runtime; see §9.3.

Chapter 2

What this system is

Akshar is an input method engine that converts Roman-script typing into Devanagari, targeted at Nepali. You type `namaste` and it offers नमस्ते.

Three properties define the design:

No neural network at runtime. The model is an EM-trained source-channel model over orthographic syllables, a Kneser-Ney syllable language model, and a linear discriminative reranker. Inference is beam search plus dot products. There is no tensor library, no GPU, and no runtime dependency beyond the Rust standard library and `serde`.

Small enough to ship in a web page. The desktop container is 11.37 MB; the browser profile is 8.91 MB raw and 4.94 MB Brotli-compressed.

Fast enough to run on every keystroke. A suggestion query costs **0.63–0.82 ms** on a desktop CPU with a beam width of 64 (§10).

2.1 What it is not

It is not a general Indic transliteration system — it is trained and measured on Nepali only. It does not beat neural baselines on named entities (§9.2). And it does not yet reach the 90% native accuracy that its error analysis shows is attainable (§12).

2.2 Measured performance

On the held-out AI4Bharat Aksharantar Nepali test split (4,101 cases), measured 2026-09-06 on `data/akshar.model` at default settings:

Split	<i>n</i>	top-1	top-5
AK-Freq (native words)	2,108	81.83%	92.22%
AK-NEI (named entities)	1,176	47.79%	69.81%
AK-NEF (named entities)	817	31.21%	53.00%
All cases	4,101	61.98%	77.98%

Bootstrap 95% CI on the pooled top-1 is [60.61%, 63.36%]; MRR is 0.6906. Character error rate on AK-Freq top-1 is 3.90%.

For reference, **IndicXlit** (AI4Bharat; an ~11M-parameter transformer) reports 80.25% top-1 on the native split and 52.67% on named entities. Native accuracy here is comparable; named-entity accuracy is substantially behind.

Resource	Desktop	Browser
Container size	11.37 MB	8.91 MB
Brotli-compressed	6.72 MB	4.94 MB
Query latency	0.63–0.82 ms	not re-measured since 2026-09-06
Cold start	~1.5 s	not re-measured

Chapter 3

Quick start

3.1 Build and install (Linux / IBus)

```
make release      # builds libakshar_ime.so and the IBus C engine
sudo make install # installs to /usr/lib/ibus/engines and /usr/share
make restart-ibus # NOT as root
```

Then add “Akshar Devanagari” as an input source in your desktop settings.

3.2 Use from Rust

```
use akshar_ime::ImeEngine;

let mut engine = ImeEngine::new(); // loads data/akshar.model
let suggestions = engine.get_suggestions("namaste", 5);
for (devanagari, score) in &suggestions {
    println!("{devanagari} {score}");
}

// Teach the engine what the user actually picked.
engine.user_confirms("namaste", "□□□□□□");
```

3.3 Use from the browser

```
make wasm      # builds wasm/pkg + the JS wrapper
make wasm-serve # serves the demo at http://localhost:8000/web/
```

```
import { createEngineFromModel } from './akshar.js';
const engine = await createEngineFromModel('/data/akshar_wasm.model');
const suggestions = engine.get_suggestions('namaste', 5);
```

See Chapter 10 for deployment detail.

Chapter 4

Problem formulation

4.1 The task

Given a Roman string R typed by a user, produce a ranked list of Devanagari strings D that the user plausibly meant. This is *transliteration*, not translation: the output should be the same word in a different script.

The difficulty is that Roman input for Devanagari is **not a code**. It is a lossy, inconsistent, user-invented approximation:

- Vowel length is routinely dropped. nepa`l`i and nepaa`l`ee are both नेपाली.
- Retroflex/dental distinctions collapse. `t` may be त or ट.
- Aspiration is inconsistent. `kh` may be ख, but `k` sometimes is too.
- The inherent schwa is written or omitted at the user’s discretion: kama`l`, kama`l`a, and km`l` all target कमल.
- Conjuncts have no standard Roman form.

So the mapping is many-to-many in both directions, and the model must learn the conventions from data rather than encode them by hand.

4.2 The source-channel decomposition

Following Li, Zhang & Su (ACL 2004), *A Joint Source-Channel Model for Machine Transliteration*, we model the user as a noisy channel: they have a Devanagari word D in mind and emit a Roman rendering R of it. Then

$$\hat{D} = \arg \max_D P(D | R) = \arg \max_D \underbrace{P(R | D)}_{\text{channel}} \cdot \underbrace{P(D)}_{\text{source}}.$$

The two factors are learned separately:

- $P(R | D)$ — the **transliteration model**, learned by EM over an unaligned parallel lexicon (Chapter 5).
- $P(D)$ — a **language model** over aksharas, smoothed with modified Kneser-Ney (Chapter 6).

Working in negative logs, the decoder minimises

$$\text{cost}(D) = \underbrace{-\log P(R | D)}_{\text{emit}} + \lambda \cdot \underbrace{-\log P(D)}_{\text{lm}},$$

with λ (`DecoderConfig::lm_weight`, default 1.0) a tunable balance. The two terms are accumulated **separately** all the way through decoding, because the discriminative reranker (Chapter 7) consumes them as independent features.

4.3 The unit of modelling: the akshara

Neither characters nor whole words are the right granularity.

Characters are wrong because Devanagari is an *abugida*: a consonant carries an inherent vowel, and a matra (vowel sign) modifies the preceding consonant rather than standing alone. The codepoint sequence क + ि is one pronounceable unit, कि, and splitting it produces meaningless states.

Whole words are wrong because the vocabulary is open — Nepali is agglutinative, and compounds and case-marked forms are productive.

So the unit is the **akshara**, the orthographic syllable of Brahmic scripts:

akshara := (consonant halanta)* consonant? (matra | independent-vowel) (anusvara | visarga | chandrabinu | nukta)*

Implemented in `src/core/akshara.rs`. Examples:

Word	Aksharas
नमस्ते	न म स्ते
क्ष्त्र	क्ष्त्र (one conjunct)
अग	अ ग
काठमाडौ	का ठ मा डौ

A halanta (virama, ँ) glues the following consonant into the current akshara, which is what makes conjuncts single units. Zero-width joiners are preserved when attached to viramas, so Nepali eyelash-ra ळ keeps its visual form.

The trained model has **16,556 aksharas** and **101,010 Roman chunks** before pruning; after pruning to those reachable from the corpus vocabulary, 5,938 emission rows remain.

4.4 Alignment is latent

The training data is a list of (R, D) pairs with **no alignment** between Roman substrings and aksharas. `kathmandu / काठमाडौ` does not say that `kath` corresponds to `का ठ`.

So the model treats the segmentation as a latent variable and sums over all of them:

$$P(R | D) = \sum_{\text{segmentations } s_1 \dots s_n \text{ of } R} \prod_{j=1}^n P(s_j | a_j),$$

where $a_1 \dots a_n$ are the aksharas of D and each s_j is a contiguous (possibly empty) chunk of Roman characters, of length at most `MAX_CHUNK = 5`. This sum is computed exactly by dynamic programming, and EM maximises it (Chapter 5).

Chapter 5

The transliteration model and EM training

Implemented in `src/core/em_trainer.rs`, `src/core/alignment.rs`, and `src/core/translit_model.rs`.

5.1 Parameters

The model is a single table:

$$\theta_{a,s} = P(s \mid a), \quad \sum_s \theta_{a,s} = 1 \quad \forall a,$$

the probability that akshara a is written as Roman chunk s . Chunks are lowercase ASCII of length 0 ... 5; the empty chunk lets an akshara be written as nothing at all, which is how the model handles dropped schwas.

Chunks are packed into a u32 for allocation-free lookup: 5 bits per character (26 letters plus an escape) in bits 0–24, and a 4-bit length in bits 26–29 (`translit_model.rs::pack_chunk_bytes`).

5.2 Initialisation

EM is sensitive to its starting point, so emissions are seeded from a deterministic codepoint aligner (`alignment.rs::align_emissive`) rather than from a uniform distribution. For each pair, the aligner walks both strings and assigns each akshara the Roman characters that plausibly correspond to it.

Bare consonants additionally seed two variants:

- the chunk plus a (the inherent schwa written out: $\text{च} \rightarrow \text{cha}$), and
- the chunk minus a trailing a (the schwa dropped),

so that EM has both conventions available from the first iteration rather than having to discover one from a distribution that assigns it zero mass.

Seed counts are normalised and floored at 10^{-4} so no observed chunk starts with probability zero.

5.3 The E-step: scaled forward-backward

For a pair with Roman length m and n aksharas, define

$$\alpha_j(i) = P(\text{first } i \text{ Roman chars generated by first } j \text{ aksharas}),$$

$$\beta_j(i) = P(\text{Roman chars } i+1 \dots m \text{ generated by aksharas } j+1 \dots n).$$

with the recurrences

$$\alpha_j(i) = \sum_{l=0}^{\min(5, i)} \alpha_{j-1}(i-l) \theta_{a_j, R[i-l..i]}, \quad \beta_j(i) = \sum_{l=0}^{\min(5, m-i)} \theta_{a_{j+1}, R[i..i+l]} \beta_{j+1}(i+l),$$

$\alpha_0(0) = 1, \beta_n(m) = 1$. The total likelihood is $Z = \alpha_n(m)$, and the expected count of (a_j, s) at position i is

$$\gamma_j(i, l) = \frac{\alpha_{j-1}(i-l) \theta_{a_j, R[i-l..i]} \beta_j(i)}{Z}.$$

5.3.1 Why the passes are scaled

Computed naively, Z underflows. A 12-akshara word whose emissions average 0.05 gives $Z \approx 2.4 \times 10^{-16}$. An earlier implementation guarded this with `if z < 1e-12 { continue; }` — a threshold 296 orders of magnitude above the f64 subnormal limit — which silently **discarded every long or flat-emission word from training**, biasing EM toward short easy words.

The fix is Rabiner-style scaling. Each forward column is divided by its own maximum c_j , and **the same factors** are divided out of the backward pass, so that with

$$F_j = \alpha_j / \prod_{t \leq j} c_t, \quad B_j = \beta_j / \prod_{t > j} c_t,$$

the scale products telescope and the posterior needs only a single per-column correction:

$$\gamma_j(i, l) = \frac{F_{j-1}(i-l) \theta B_j(i)}{c_j \cdot F_n(m)}.$$

Derivation: $\alpha_{j-1} = F_{j-1} S_{j-1}$ and $\beta_j = B_j S_n / S_j$ with $S_j = \prod_{t \leq j} c_t$, and $Z = F_n(m) S_n$. Substituting, the S factors cancel to $1/c_j$ because $S_j = S_{j-1} c_j$. $\square$

This is verified by test, not by inspection: `scaled_forward_backward_matches_unscaled` compares against an unscaled reference implementation, `posteriors_sum_to_weight_per_akshara` checks that each akshara's posterior mass equals its observation weight, and `long_low_probability_word_is_not_dropped` pins the 12-akshara case the old floor discarded.

The E-step is embarrassingly parallel over pairs and runs on scoped threads.

5.4 The M-step

Counts are normalised with Dirichlet smoothing toward the global chunk unigram $u(s)$, so that a chunk seen with one akshara does not receive zero probability under another:

$$\theta_{a,s} \leftarrow \frac{\text{count}(a, s) + \alpha u(s)}{\sum_{s'} \text{count}(a, s') + \alpha}, \quad \alpha = 0.05.$$

Twelve iterations are run by default. EM over all 3,588,793 pairs takes about 170 s on a desktop CPU.

5.5 What the model looks like when trained

Emissions are stored as `Vec<Vec<(chunk_id, -log P)>>`, one row per akshara, each row **sorted ascending by chunk id**. That ordering is not incidental: the container delta-encodes the ids (§8.2) and the runtime binary-searches them (§10.2), and both would be incorrect for any other order.

Chapter 6

The language model

Implemented in `em_trainer.rs::build_kn_lm` and consumed by `translit_model.rs`. This is an akshara n -gram model, and it is **the largest single contributor to accuracy** in the system: removing the trigram order costs 3.89pp of native top-1 (§9.1).

6.1 Why Kneser-Ney

Maximum-likelihood n -gram estimates assign zero to unseen contexts, and add- α smoothing is badly wrong on large alphabets — with 16,556 aksharas, a single observation under add-1 would imply $P \approx 0.67$.

Kneser-Ney addresses a subtler problem. The right lower-order estimate is not “how often did this akshara occur” but “**in how many distinct contexts** did it occur”. An akshara that appears constantly but always after the same predecessor (the *Kong* in *Hong Kong*) is a poor bet in a novel context. The continuation probability captures exactly that:

$$P_{\text{cont}}(b) = \frac{|\{a : c(a, b) > 0\}| + 0.5}{|\{(a, b) : c(a, b) > 0\}| + 0.5N},$$

with the $+0.5$ floor keeping the log finite for aksharas that only ever appear word-initially. N is the akshara vocabulary size.

6.2 Modified Kneser-Ney discounts

A single absolute discount δ over-discounts frequent n -grams and under-discounts singletons. Chen & Goodman (1999) use three discounts, chosen by the count of the n -gram, estimated from the counts-of-counts $n_1 \dots n_4$:

$$Y = \frac{n_1}{n_1 + 2n_2}, \quad D_1 = 1 - 2Y \frac{n_2}{n_1}, \quad D_2 = 2 - 3Y \frac{n_3}{n_2}, \quad D_3 = 3 - 4Y \frac{n_4}{n_3},$$

subject to $0 \leq D_i \leq i$. That constraint matters. An earlier implementation clamped all three to $\leq 0.9/0.9/0.95$; the values this corpus actually produces are

order	D_1	D_2	D_3
bigram	0.6049	1.0362	1.4443
trigram	0.6696	1.0949	1.4330

so D_2 was being cut by 13% and D_3 by 34%, both pinned at the ceiling. The effect was to collapse modified Kneser-Ney back into single-discount absolute discounting at $d \approx 0.9$ — *worse* than the fixed 0.75 it replaced. The bounds are now correct. See §9.4 for what fixing them was worth.

If n_1, n_2 or n_3 is zero the estimator is degenerate and the model falls back to fixed (0.5, 0.75, 0.95).

6.3 The interpolated model

For a bigram context a with total count $c(a) = \sum_b c(a, b)$:

$$P(b | a) = \frac{\max(c(a, b) - D_{c(a, b)}, 0)}{c(a)} + \lambda(a) P_{\text{cont}}(b), \quad \lambda(a) = \frac{\sum_b D_{c(a, b)}}{c(a)}.$$

$\lambda(a)$ is exactly the mass removed by discounting, so the distribution sums to one. The trigram has the same form with context (a, b) and $c(a, b)$ as the denominator, backing off to the bigram.

A **word-start prior** is estimated separately from word-initial counts:

$$P(a | \#) = \frac{c_{\#}(a) + 0.5}{W + 0.5N}, \quad W = \text{corpus word count}.$$

6.4 Storage and lookup

All quantities are stored as $-\log$ weights in f32. Per context the model stores the seen successors and one backoff weight $-\log \lambda$; an unseen successor costs $-\log \lambda + (-\log P_{\text{lower}})$, which reconstructs the interpolated value exactly. `trigram_weight` falls back to `bigram_weight`, which falls back to `unigram_kn`.

Successor rows are sorted ascending by id and looked up by **binary search**. This is not a micro-optimisation: the beam performs roughly 50,000 LM lookups per query, and when these rows were scanned linearly they accounted for most of the engine’s runtime (§10.2).

6.5 Two known deviations from textbook KN

Both are recorded here rather than hidden, and both are open (§11):

1. **The trigram interpolates against the highest-order bigram**, not the continuation estimate $N_{1+}(\bullet, b, c)$ that Kneser-Ney requires. The bigram→unigram step is correct; the trigram→bigram step is not.
2. **There is no end-of-word symbol**. The model has a word-start prior but no `</w>`, so trigram mass is deficient by exactly the word-final fraction, and the model cannot express that a given akshara is an implausible way to end a word. Since 51.9% of native top-1 errors are matra-only and matra errors concentrate word-finally, this is a real gap and not merely a formal one.

Chapter 7

Decoding

Implemented in `src/core/decoder.rs`.

7.1 The lattice

Decoding is a shortest-path search over a lattice built on the Roman string. Nodes are character positions $0 \dots m$. An edge from position p to $p + l$ is labelled with an akshara a that can emit the chunk $R[p..p+l]$, and carries weight $-\log P(R[p..p+l] \mid a)$.

Edges come from a **reverse index** built once at load time: chunk $\rightarrow$ list of $(akshara, weight)$, sorted by weight and truncated. Two caps keep the lattice tractable:

- `max_emission_weight = 8.0` — emissions worse than e^{-8} are alignment noise and are dropped.
- `max_aksharas_per_chunk = 16` — at most 16 aksharas compete for any chunk.

A path from 0 to m spells one candidate Devanagari string; its cost is the sum of edge weights plus the LM cost of its akshara sequence.

7.2 Beam search

The search is step-synchronous over aksharas. Each step expands every beam state across every edge available at its position, scores the extension, and keeps the best `beam_width` hypotheses (default 64, `AKSHAR_BEAM`).

A state carries its position, its previous two aksharas (the trigram context), the running emit and lm costs **separately**, a path hash, and an index into a persistent path arena. Extending a path is $O(1)$: one arena cell holding $(parent, akshara)$. Reconstruction walks parents backwards at the end.

Three properties of this loop are deliberate and worth stating, because each looks like an oversight:

Distinct paths are not merged by state. A conventional Viterbi beam would collapse hypotheses sharing $(pos, prev2, prev)$ and keep the best. This decoder extracts *k-best paths*, not the 1-best path per state, so merging would destroy the diversity the reranker needs. An earlier version keyed a merge map on $(pos, prev2, prev, path_hash)$, which could only ever merge on a hash collision — it built and tore down a hash map every step to do nothing, and was removed.

Pruning uses `select_nth_unstable_by`, not a sort. Keeping the best 64 of ~5,000 hypotheses does not require ordering them; the beam's internal order is never read. This is $O(n)$ instead of $O(n \log n)$.

Arena cells are materialised only for survivors. Hypotheses are generated into a lightweight `Cand` struct holding the parent index and its own akshara; only after pruning are the survivors written into the arena.

This cuts arena writes from $O(\text{beam} \times \text{edges})$ to $O(\text{beam})$ per step.

Completed paths (those reaching position m) are collected into a map keyed by the output string, keeping the best cost per string, and sorted once at the end.

7.3 Two passes: free and trie-constrained

`decode_union` runs the beam **twice** and merges:

1. **Free decode** (`decode_detailed`) — the full lattice. Can produce any akshara sequence, including words that do not exist. This is what handles novel compounds and out-of-vocabulary names.
2. **Trie-constrained decode** (`decode_in_words_detailed`) — the lattice intersected with a word trie built from the corpus vocabulary. Every edge must extend a valid word prefix, and only trie terminals are returned.

The second pass is far cheaper (0.12 ms against 0.55 ms) because most akshara sequences are not word prefixes, so its beam collapses almost immediately. It is also strictly a recall addition: it surfaces real words the free beam ranked outside its top 50.

Its contribution is +0.66pp top-1 and +1.14pp top-5 (§9.1). Running *only* the trie pass would be 4x faster still, but costs 15.65pp of native top-1 — the vocabulary simply does not cover the test set — so both passes stay.

7.4 What the decoder returns

A `DecodedCandidate` per output string, carrying `emit`, `lm` and `akshara_count` **separately** rather than as one summed score, because the reranker uses them as independent features.

Chapter 8

Discriminative reranking

Implemented in `src/core/reranker.rs`, weights in `reranker_weights.rs`.

The decoder’s generative score is not the last word. It knows nothing about how common a word is, what shape Nepali words have, or how the language’s morphology works. The reranker re-scores the decoder’s top candidates using features the generative model cannot express.

8.1 The baseline heuristic

Before any learned model, candidates are scored by a three-parameter heuristic:

$$h(D) = \text{emit} + 0.85 \cdot \text{lm} - 0.75 \cdot \log(1 + f(D)),$$

where $f(D)$ is the corpus frequency of D . This alone achieves **81.02%** native top-1 — it is a strong baseline, and the learned model must be measured against it, not against nothing.

8.2 Dense features

29 features per candidate (`extract_dense_features`):

Index	Feature
0–1	emit, lm from the decoder
2	akshara count
3	decoder rank
4–5	heuristic score, heuristic rank
6–8	$\log(1 + f)$, frequency-rank percentile, in-vocabulary indicator
9	output length in characters
10	total matra count
11–20	per-matra counts (10 matras)
21–23	nasals, visarga, halants
24–26	vowel-initial, ends-in-matra, ends-in-nasal/visarga
27	Roman input length
28	morphology-aware effective log frequency

Feature 28 deserves a note. Nepali is agglutinative, so an inflected form may be absent from the corpus while its stem is frequent. `morph_effective_log_freq` strips one of 34 known suffixes (को, हरू, लाई, एको, ...) and, if the stem has frequency ≥ 5 , returns $\log f(\text{stem}) + 3.5$. This gives unseen but well-formed inflections a frequency prior instead of zero.

Features are standardised before scoring:

$$\text{score}_{\text{dense}}(D) = \sum_{k=0}^{28} w_k \cdot \frac{\phi_k(D) - \mu_k}{\sigma_k}.$$

μ and σ **travel with the model**, in the v5 container. They used to be compiled-in constants that no training stage refreshed — and since retraining the EM/LM shifts the `emit` and `lm` distributions (a 100k retrain moved `lm` from $\mu = 21.83, \sigma = 5.74$ to $24.64, 7.03$), the weights were being applied to differently-scaled inputs with nothing to detect it. Containers older than v5 fall back to the constants.

8.3 Sparse lexicalized features

Seven templates are hashed into a 2^{20} -slot table of 18 weights:

1. length-delta bucket (aksharas minus Roman characters)
2. final akshara $\times$ final Roman character
3. first akshara $\times$ first Roman character
4. matra $\times$ preceding consonant
5. morphological suffix $\times$ Roman tail character
6. final matra $\times$ final Roman character
7. penultimate consonant $\times$ final matra

Weights are quantised to 18 with the scale set from the 99.9th percentile of non-zero magnitudes and explicit clipping — not from the single largest weight, which previously let one outlier compress every other weight into a handful of levels.

8.4 The blend

$$\text{score}(D) = (1 - \gamma) \cdot (-z(h(D))) + \gamma \cdot z(\text{score}_{\text{dense+sparse}}(D)), \quad \gamma = 0.3,$$

where $z(\cdot)$ standardises within the candidate list. γ was tuned, and is worth understanding honestly:

γ	meaning	AK-Freq top-1
0.0	heuristic only, learned model unused	81.02%
0.3	shipped	81.83%
0.6		81.31%
1.0	learned model only	76.47%

The learned model used alone is 5.36pp worse than the three-parameter heuristic. $\gamma = 0.3$ is not a cautious discount of a good model; it is the blend point at which a weak model stops doing damage.

The cause is identified in §12.3: `W_DENSE` has never been retrained by this pipeline. It is a frozen constant, and the generator named in its header does not exist in the repository. Training more sparse capacity on top of misfitted dense weights was tested at 5x the data and did not help.

8.5 The cascade

Full feature extraction — akshara segmentation, matra scans, sparse hashing — is far more expensive than the heuristic, and candidates the heuristic already ranks far down never reach the top of the final list. Only the top AKSHAR_RERANK_DEPTH (default 24) by heuristic rank are fully scored; the rest keep heuristic order below the scored block.

Measured on AK-Freq: depth 24 gives 81.83/92.22, depth 50 (no cascade) gives 81.69/92.22, depth 12 gives 81.78/92.41, depth 6 gives 81.31/91.98. The effect is within noise down to depth 12.

One subtlety is documented in the source: the synthetic scores given to unscored candidates participate in the γ z-blend, so the blend weight moves slightly with the scored/unscored ratio. Ordering among scored candidates is unaffected (z-scoring is affine).

Chapter 9

The engine: candidate fusion and learning

Implemented in `src/core/engine.rs`.

9.1 Sources of evidence

The engine merges candidates from several sources into one ranked list, taking the **maximum** evidence per output string:

Source	Score	Notes
Decoder + reranker	$800,000 / (1 + \text{cost})$	cost is the reranker margin below the best
User-confirmed trie	$900,000 + \text{freq}$	prefix match on learned words
User fuzzy (SymSpell)	$50,000 - 12,000 \cdot d$	distance-verified, learned words only
Context re-rank	multiplicative	user bigrams

This score space is the weakest part of the architecture, and this manual does not pretend otherwise.

The bands are hand-tuned u64 constants compared against a hyperbolically squashed reranker score, and two of the three serious defects found in this system came from that design:

- A corpus-wide fuzzy source scored at 850,000 — above the decoder band — so any fuzzy hit displaced the reranker’s top-1. It cost **30.8pp** of native top-1 and, when tested fairly, contributed no recall at any band. It was removed.
- The user fuzzy band, at 38,000 for a distance-1 match, sits *below* the decoder’s eighth-ranked candidate (~250,000). A word the user has confirmed cannot be recovered from a typo. The path is structurally unreachable (defect D18, §11).

The correct fix for both is not a better constant but a single log-linear score in which every source contributes a *feature* rather than occupying a band. That work is §12.

9.2 Digits and punctuation

Digits map to Devanagari numerals (123 → १२३) and a trailing . offers purnabiram (namaste . → नमस्ते ।). These are handled directly, before the statistical path.

9.3 Adaptive learning

`user_confirms(roman, devanagari)` records the user's choice into:

- the **trie** (`src/core/trie.rs`) — prefix lookup with frequency;
- the **SymSpell index** (`src/fuzzy/symspell.rs`) — delete-variant map for typo tolerance;
- the **context model** (`src/core/context.rs`) — user bigrams for re-ranking by preceding word.

Learned state is separate from the model and persists via `src/persistence.rs` (desktop) or `export_state/import_state` (browser).

9.3.1 How fuzzy matching actually behaves

Two mechanisms remain, and they are different in kind:

1. **Query-variant rewriting** (`src/core/normalizer.rs`) — a Dijkstra search over a small rewrite transducer ($ee \rightarrow ii$, $oo \rightarrow uu$, $w \rightarrow v$, separator removal), each rule carrying a cost. Cold-start: needs no learning. Measured contribution on the benchmark: **0.00pp**, because only the first variant is ever decoded and the EM emissions already absorb these alternations.
2. **User SymSpell** — symmetric-delete matching over confirmed words. Every hit is verified with bounded Damerau-Levenshtein before scoring, because a delete-set intersection is a *necessary* condition only: at `max_edit_distance = 1` it still returns pairs at true distance 2.

Behaviour is pinned by `tests/fuzzy_behavior.rs`.

Chapter 10

The model container

Implemented in `src/core/unified.rs` and `src/core/codec.rs`.

10.1 One file

The engine loads exactly one artefact, `data/akshar.model`. It holds:

- the transliteration model (aksharas, chunks, emissions),
- the Kneser-Ney LM (unigram continuation, bigrams, trigrams, backoffs, word-start prior),
- the corpus vocabulary with frequencies,
- the sparse reranker table and its dequantisation scale,
- the dense-feature normalisation statistics (μ , σ).

10.2 Versions

Version	Change
v1	initial layout
v2	adds <code>sparse_scale</code>
v3	compact codec: CSR adjacency, delta varints, 8-bit codebooks
v4	removes the word-bigram table (19.5 MB for +0.16pp)
v5	carries the reranker's dense normalisation statistics

`save` writes v5; v1–v4 still load, with older containers falling back to the compiled-in normalisation constants. `bincode` is positional and not self-describing, so each version has its own struct and the reader dispatches on a version peeked from the header — appending a field to an existing struct would silently corrupt every file already written with it.

10.3 Compact encoding

Naive `bincode` of these structures is 66.59 MB. Four techniques bring it to 11.37 MB with no accuracy change:

- **CSR adjacency with delta varints.** Successor ids within a row are ascending, so only the gaps are stored, as variable-length integers. (This is also why the rows must stay sorted — see §5.5 and §10.2.)

- **8-bit weight codebooks.** Each numeric section gets a 256-entry codebook of f32 values; entries store an index. Weights cluster tightly, so the quantisation error is far below the model's discrimination threshold.
- **Front-coded vocabulary.** Words are stored as akshara-id sequences against a shared prefix.
- **Packed chunk strings.**

Section sizes can be inspected at any time:

```
cargo run --release --bin probe_model -- data/akshar.model --inspect
```

10.4 Browser profile

`make web-model` produces `data/akshar_wasm.model` by relative-entropy pruning of the trigram LM. `TRIGRAM_THRESHOLD` (default `3e-2`) trades size against accuracy along a measured curve; at the default it keeps 47% of trigram transitions and yields 8.91 MB raw, 4.94 MB Brotli.

Chapter 11

Training

Implemented in `src/bin/train/train.rs`. One command produces the container:

```
make train-quick    # reranker on 100k pairs, ~10 min
make train-mid      # reranker on 500k pairs, ~40 min  <- validation gate
make train-full     # reranker on all 3.59M, ~4 h
```

11.1 What `--reranker-pairs` does and does not control

It sizes the reranker’s training set only. The EM emission model, the Kneser-Ney LM and the corpus vocabulary always consume the full data regardless of which target is run:

Component	Data used, every run
EM emissions + KN LM	all 3,588,793 parallel pairs
Vocabulary	470,012 words from a 1.5 GB corpus (hapax pruned at $f < 3$)
Reranker sparse table	<code>--reranker-pairs</code>

This matters when interpreting a quick run: it exercises the EM and LM changes at full scale, but not the reranker’s batching.

11.2 The four phases

Phase 1 — EM. Ingest pairs, seed emissions from the aligner, run 12 EM iterations, build the Kneser-Ney LM. ~170 s.

Phase 2 — Vocabulary. Stream the running-text corpus, count word frequencies, prune below $f = 3$. ~33 s.

Phase 3 — Reranker. Decode each training pair, extract dense and sparse features, and train the sparse table by softmax cross-entropy over the candidate list with AdaGrad. Above 200,000 pairs this runs **chunked**: batches of 100,000 are decoded once and reused for all epochs, which is why a full run is ~4 h rather than ~18 h.

Phase 4 — Pack. Prune emission rows unreachable from the vocabulary, quantise the sparse table, and write the v5 container.

11.3 Learning rate

AdaGrad adapts per slot, so the global schedule only needs a gentle anneal: `lr` starts at 0.05 and decays to 10% of that across the whole run, spread evenly over the batches.

This replaced a schedule that compounded $0.8^{(\text{epochs}-1)}$ *per batch* on top of a per-batch 0.995 — a factor of 0.6368 per batch, reaching 4.4×10^{-9} by batch 36. A `train-full` run under that schedule trained effectively on the first ~500k pairs and no-opped the remaining ~3M, which is why adding data had never helped.

11.4 Held-out monitoring

4,000 pairs are held out from the tail of the corpus, decoded once, and scored after every batch (chunked) or epoch (non-chunked). The table written into the container is the **best by dev loss**, not necessarily the last.

This is not optional hygiene. The sparse table has $\sim 10^6$ parameters and no regularisation, so overfitting is the default failure mode — and before this, only training loss was reported, which cannot distinguish learning from memorisation.

Watch for: dev loss falling monotonically, and the final learning rate within an order of magnitude of the first.

Caution. `train` writes `data/akshar.model` unless `--out` is given, so a training run silently replaces the released container. Back it up, or always pass `--out`, before experimenting.

11.5 Data

Artefact	Contents
<code>data/aksharantar/train_devanagari.jsonl</code>	3.59M Roman/Devanagari pairs
<code>data/aksharantar/valid_devanagari.jsonl</code>	held-out validation split
<code>data/aksharantar/test_devanagari.jsonl</code>	4,101-case test split
<code>data/store/corpus_clean.txt</code>	1.5 GB Nepali running text
<code>data/eval/test_multiref.jsonl</code>	14,410 loose romanizations of the test set

Held-out text never contributes to vocabulary counts or the EM model.

Chapter 12

Evaluation

12.1 Harnesses

Command	What it reports
<code>make eval</code>	top-1/top-5 per Aksharantar split
<code>make eval-full</code>	bootstrap 95% CIs, MRR, per-query latency
<code>make eval-errors</code>	oracle curves, error taxonomy, CER, collision bound
<code>cargo test --release</code>	unit tests plus the accuracy regression guard

Isolated-word accuracy does not see everything, so two further harnesses exist: `evaluate_sentences` measures word accuracy *in context* on held-out running text, and `evaluate --dataset data/eval/test_multiref.jsonl` measures tolerance of loose romanization across 14,410 alternative spellings.

12.2 The accuracy regression guard

`tests/accuracy_regression.rs` evaluates a 400-case AK-Freq sample on every `cargo test`, failing below 76% top-1 or 88% top-5.

It exists because a **30.8pp regression once shipped through 92 green unit tests**. Every component was individually correct; their *composition* was wrong. No amount of unit testing detects that, and this manual would rather state the lesson than the coverage number.

12.3 Headroom, and where the errors are

From `make eval-errors` at beam 256:

	AK-Freq	All
engine top-1 (strict / multi-ref)	81.83% / 82.16%	61.89% / 62.23%
decoder oracle @2	89.8%	71.4%
decoder oracle @5	92.4%	78.1%
decoder oracle @50	94.3%	85.1%
matra-only share of misses	51.9%	37.2%

	AK-Freq	All
top-1 if the matra class were solved	91.03%	76.08%
CER (engine top-1)	3.90%	11.66%

Three things follow, and they set the roadmap:

1. **The gap is ranking, not generation.** The correct answer is in the decoder’s top 50 for 94.3% of native cases but ranked first for only 81.8%. 12.5pp are being generated and then mis-ranked.
2. **Most of that is a binary decision.** Oracle@2 is 89.8%, so 8.0 of those 12.5 points are a choice between the top two candidates — a far better-posed problem than a 50-way ranking.
3. **Half the errors are vowel signs.** 51.9% of native misses are *matra-only*: prediction and gold agree after stripping vowel-length and nasal marks. Solving that class alone would reach 91.03%.

12.4 The collision bound

`analyze_errors` also computes the ceiling for **any** string-only system whose sole prior is corpus unigram frequency: for each Roman input, could the most frequent Devanagari word consistent with it be the gold answer?

$$\text{Acc}^* = \frac{1}{N} \sum_i \mathbb{1} \left[D_i^{\text{gold}} = \arg \max_{D \in A(R_i)} f(D) \right] = 99.15\%$$

Only 0.85% of cases are unwinnable (1.6% of Roman inputs are genuinely ambiguous). **The dataset is not the constraint.**

12.5 Statistical method

Accuracy differences between configurations are compared with **McNemar’s test** on paired per-case outcomes (`examples/ablate_paired.rs`), two-sided, exact binomial on the discordant pairs.

Comparing two independent accuracy numbers and their confidence intervals understates the evidence when both systems see the same cases; what matters is whether the cases where they *disagree* lean one way. Reported as w->r (full system wrong, ablated right) and r->w (the reverse).

Chapter 13

Ablations: what each component is worth

Every component here can be switched off at runtime, so this table is reproducible from the shipped binary rather than from patched builds.

13.1 Switches

Variable	Effect
AKSHAR_NO_TRIE_UNION=1	skip the trie-constrained decode pass
AKSHAR_TRIE_ONLY=1	skip the free lattice beam
AKSHAR_NO_SPARSE=1	drop the 2^{20} sparse reranker table
AKSHAR_GAMMA=<f>	override the dense/heuristic blend
AKSHAR_NO_TRIGRAM=1	force the LM to back off to bigrams
AKSHAR_NO_VARIANTS=1	decode the raw query only
AKSHAR_BEAM=<n>	beam width (default 64)
AKSHAR_RERANK_DEPTH=<n>	cascade depth (default 24)

13.2 Contribution to native top-1

Full system: **81.83%** on AK-Freq.

Component removed	top-1	Δ
trigram LM (bigram only)	77.94%	-3.89
dense reranker ($\gamma = 0$)	81.02%	-0.81
trie-constrained pass	81.17%	-0.66
sparse table (2^{20})	81.93%	+0.09
corpus lexicon	81.83%	0.00
query variants	81.83%	0.00

The **language model is the engine of this system**. Everything else is comparatively marginal.

13.3 Paired significance

Marginal deltas are not evidence on their own. McNemar’s test on paired outcomes:

Removing the sparse table ($\sim 10^6$ parameters, 1.00 MB, 8.8% of the container):

Split	n	full	ablated	Δ	w $\rightarrow$ r	r $\rightarrow$ w	p
AK-Freq	2108	81.83%	81.93%	+0.09	15	13	0.851
AK-NEF	817	31.21%	29.38%	−1.84	3	18	0.0015
AK-NEI	1176	47.79%	46.85%	−0.94	13	24	0.099
ALL	4101	61.98%	61.40%	−0.59	31	55	0.013

Removing the dense reranker ($\gamma = 0$):

Split	n	full	ablated	Δ	w $\rightarrow$ r	r $\rightarrow$ w	p
AK-Freq	2108	81.83%	81.02%	−0.81	15	32	0.019
AK-NEF	817	31.21%	30.23%	−0.98	15	23	0.256
AK-NEI	1176	47.79%	46.17%	−1.62	18	37	0.015
ALL	4101	61.98%	60.91%	−1.07	48	92	0.0003

Two conclusions:

- The **sparse table does nothing on native words** ($p = 0.851$; the +0.09 is noise). Its entire measurable value is named entities. A paper reporting only native accuracy cannot justify 10^6 parameters for it.
- The **dense reranker is real but small**: −1.07pp pooled at $p = 0.0003$.

They are not independent. “Remove both” reproduces “remove dense” *exactly* — identical accuracies and identical discordant counts on every split — because at $\gamma = 0$ the blend returns $-h(D)$ and the sparse contribution is computed and then discarded. Sparse acts only *through* dense. Reporting them as two independent contributions would be wrong.

13.4 Components that were removed

Three subsystems were measured, found to contribute nothing, and deleted.

Corpus-wide SymSpell. Indexed a romanization of the top 100k corpus words and scored hits at 850,000 — above the decoder band. Cost **−30.79pp** native top-1 (81.74% $\rightarrow$ 50.95%), NEI 47.62 $\rightarrow$ 25.68, NEF 31.21 $\rightarrow$ 20.81. Given a fair test (distances verified, per-edit penalty) it still cost 9.6pp at a competing band, and at any safe band produced results *byte-identical* to being switched off. It never contributed recall at any setting.

Corpus lexicon. Dead by construction: the shipped path never loaded one, and its data file was no longer produced by the pipeline. 0.00pp on every split.

Lattice CRF and pair model. 1,518 lines, never constructed, never packed into the container. The “modified Kneser-Ney” work of an earlier commit had landed here — in modules that never shipped.

Total removed: $\sim 1,862$ lines, no measurable accuracy change.

13.5 Modified Kneser-Ney against a single discount

Two otherwise-identical 100k runs:

LM	AK-Freq	AK-NEF
modified KN ($D_1=0.605$, $D_2=1.036$, $D_3=1.444$)	81.55% / 92.31%	30.60% / 53.00%
fixed $\delta = 0.75$	81.26% / 92.22%	30.84% / 53.86%

+0.29pp is inside one standard error (0.84pp at $n = 2108$), and fixed δ is ahead on AK-NEF. **Modified Kneser-Ney is now correctly implemented but is not measurably better than a single discount on this data.** It is retained because it is the standard estimator and free at runtime; it should not be described as an improvement.

Chapter 14

Performance

14.1 Where the time goes

Profiled over 1,000 real queries (`examples/profile_decode.rs`, mean input length 10 characters):

Phase	ms/query	share
free lattice beam	0.55	67%
trie-constrained beam	0.12	15%
decode_union (both + merge)	0.686	84%
reranker	0.047	6%
engine overhead	0.082	10%
end to end	0.816	

`make eval-full` reports **0.63–0.72 ms/query** at $k = 10$ over the full test set, run to run. Cold start is ~1.5 s.

14.2 How it got there

The engine was at 3.5 ms. Three changes, none of which cost accuracy:

Binary-search LM lookups (5.4x). `bigram_weight` and `trigram_weight` scanned their successor rows linearly. The beam expands ~5,000 hypotheses per step with one LM lookup each — roughly 50,000 linear scans per query. Rows are stored ascending by id, so this is a binary search. Free beam: 3.128 → 0.575 ms. Output was byte-identical (1944/2108 before and after).

O(n) beam pruning. `select_nth_unstable_by` instead of a full sort to keep the best 64 of ~5,000.

Lazy arena materialisation. Path cells are written only for hypotheses that survive pruning, cutting arena writes from $O(\text{beam} \times \text{edges})$ to $O(\text{beam})$ per step.

Indexed akshara lookup. `TranslitModel::akshara_id` was a linear scan over the vocabulary, called ~500,000 times while building the word trie at start-up. Cold start: 4.35 → 1.55 s.

The lesson worth carrying: the reranker was assumed to be the bottleneck and was 6% of the time. **Profile before optimising.**

14.3 Trade-offs available

Configuration	native top-1	latency
default (beam 64, both passes)	81.83%	0.67 ms
trie-constrained pass only	66.18%	0.18 ms

Trie-only decoding is 4x faster but costs 15.65pp: the corpus vocabulary does not cover the test set. Beam width and cascade depth are tunable via AKSHAR_BEAM and AKSHAR_RERANK_DEPTH.

Chapter 15

Deployment

15.1 Linux / IBus

```
make release && sudo make install && make restart-ibus
```

src/ibus_engine.c handles key events and the candidate UI, calling the Rust core through the C ABI in src/c_api.rs. The library is installed to /usr/lib, the engine to /usr/lib/ibus/engines, the component XML to /usr/share/ibus/component, and the model to /usr/share/akshar-ime.

Learned state persists to the user's data directory (src/persistence.rs); make reset-learning clears it.

15.2 Browser / WebAssembly

```
make wasm          # wasm/pkg + JS wrapper
make wasm-serve    # demo on :8000
```

```
import { createEngineFromModel } from './akshar.js';
const engine = await createEngineFromModel('/data/akshar_wasm.model');
engine.get_suggestions('namaste', 5);
engine.confirm('namaste', 'नमस्ते');
localStorage.setItem('akshar', engine.export_state());
```

Serve akshar_wasm.model with Content-Encoding: br and a long cache lifetime; it is 4.94 MB compressed and immutable.

WasmEngine::from_bytes(model, lexicon, weights) retains its lexicon parameter as an ignored no-op so existing callers keep working. Pass null.

Browser latency has not been re-measured since 2026-09-06; the last figure was ~3–7 ms per call, taken before the 4.3x decoder speed-up, so it should be expected to be substantially better.

15.3 Offline tools

Binary	Purpose
train	full pipeline, writes the container

Binary	Purpose
pack_model, repack_model	assemble / re-encode a container
prune_lm, prune_model, quantize_model	browser-profile compaction
build_wordfreq_text	vocabulary from running text
romanize	Devanagari → Roman with orthographic canonicalisation
probe_model	inspect container sections, or decode one word
evaluate, evaluate_aksharantar, evaluate_sentences	accuracy harnesses
analyze_errors	oracle curves, taxonomy, collision bound

src/fuzzy/grammar.rs supports romanize and evaluate_sentences and is not part of the runtime engine.

Chapter 16

Source map

11,094 lines of Rust. Runtime core first, then tooling.

File	Lines	Role
core/engine.rs	989	orchestration, candidate fusion, learning
core/em_trainer.rs	968	EM (scaled forward-backward), Kneser-Ney LM construction
core/codec.rs	788	compact container encoding
core/decoder.rs	635	lattice beam search, both passes
core/reranker.rs	569	dense + sparse features, blend, cascade
core/unified.rs	494	container layout and version dispatch
core/translit_model.rs	367	model access: emissions, LM lookups, backoff
core/normalizer.rs	309	query-variant rewriting
core/alignment.rs	250	deterministic aligner used to seed EM
core/akshara.rs	199	akshara segmentation
core/trie.rs	148	user-learned dictionary
core/wordtrie.rs	92	corpus vocabulary trie for the constrained pass
core/context.rs	52	user bigram re-ranking
core/holdout.rs	66	held-out split helper
fuzzy/sympell.rs	113	symmetric-delete index
fuzzy/grammar.rs	481	orthographic canonicalisation (offline tools only)
learning.rs	123	learning orchestration
persistence.rs	75	learned-state serialisation
c_api.rs	118	C ABI for IBus
wasm.rs	411	WebAssembly bindings
bin/train/train.rs	861	training pipeline
bin/evaluate/*	1,598	evaluation and error analysis

File	Lines	Role
bin/build/*	1,275	container assembly, pruning, diagnostics

Tests: tests/accuracy_regression.rs, tests/fuzzy_behavior.rs, plus module tests. Diagnostics: examples/profile_decode.rs, examples/ablate_paired.rs, examples/diag_reranker.rs, examples/diag_fuzzy.rs.

Chapter 17

Known defects and limitations

Recorded openly. Nothing here is hidden in a footnote.

17.1 Open defects

D18 — the user fuzzy path is unreachable. A learned word cannot be recovered from a typo. The path scores 50,000 — 12,000*d*, i.e. 38,000 at distance 1, while the decoder’s eighth-ranked candidate still scores ~250,000. Pinned by an `#[ignore]d` test in `tests/fuzzy_behavior.rs`, with a companion test that fails if the band gap ever closes. **The fix is not a larger constant** — that is precisely how the 30.8pp regression happened — but the log-linear fusion in §12.

The trigram’s lower-order estimate is wrong (§6.6). It interpolates against the highest-order bigram rather than continuation counts.

There is no end-of-word symbol (§6.6). Trigram mass is deficient by the word-final fraction, and the model cannot penalise implausible word endings.

The candidate-union score space is hand-tuned u64 bands. Two of the three serious defects found in this system originated there. It discards the reranker’s calibration by squashing it through $800,000/(1 + \text{cost})$.

17.2 Things that do not work as their names suggest

The discriminative reranker is worse than the heuristic when used alone (76.47% against 81.02%). Its net contribution at the tuned blend is +0.81pp on native words. With $\sim 10^6$ sparse parameters trained on a candidate-ranking objective and no regularisation, overfitting is the likely explanation.

Modified Kneser-Ney is not measurably better than a single discount (§9.4).

Query-variant rewriting contributes 0.00pp (§9.1). 309 lines of Dijkstra over a rewrite transducer whose only decoded output is the identity variant. Retained because it is the only cold-start mechanism for spelling alternation, but it is not currently earning its place.

17.3 Scope limitations

- **One language.** Trained and measured on Nepali only.
- **One test set.** Aksharantar. The Dakshina benchmark, on which IndicXlit reports its headline, is not evaluated.
- **Named entities are well behind** the neural baseline: 31–48% against 52.67%.
- **The browser profile has not been re-measured** since 2026-09-06.

- **Numbers predate a full retrain.** Every figure was produced with a reranker trained on 100k pairs; the first run to use all 3.59M has not yet been made.

Chapter 18

Roadmap

The measured decomposition of the 18.2 missing points on native words:

$$\underbrace{94.3 - 81.8 = 12.5}_{\text{ranking: generated, mis-ranked}} + \underbrace{100 - 94.3 = 5.7}_{\text{generation: never in top 50}}$$

Work is ordered by expected points per unit of effort.

- 1. Factored matra model.** 51.9% of native misses are matra-only, and solving that class alone reaches 91.03%. This is the largest single lever, and it is a better-posed problem than whole-word reranking: predict vowel-sign assignment given a consonant skeleton.
- 2. A top-2 discriminator.** Oracle@2 is 89.8%, so 8.0 of the 12.5 ranking points are a binary choice between the top two candidates. Training a model specifically on that decision is a smaller and better-conditioned problem than a 50-way ranker.
- 3. Log-linear candidate fusion.** Replace the u64 bands with a single log-linear score in which every source contributes a feature. This removes an entire defect class — both the 30.8pp regression and D18 were band-ordering bugs — and makes the sources jointly tunable.
- 4. End-of-word symbol.** Cheap, and it targets matra errors, which concentrate word-finally.
- 5. Refit the dense weights.** See §12.3 — this replaced “retrain the reranker on more data”, which was tested and falsified.
- 6. Continuation-count trigram backoff.** Textbook correctness; modest gain.

18.1 Result: more supervision was tested and does not help

The plan above once contained “retrain the reranker on all 3.59M pairs”, on the theory that a 2^{20} -parameter table trained on 100k examples was starved. That was run and **falsified**, and the negative result is more useful than the hypothesis was.

train-mid (500,000 reranker pairs — 5x the released model, with the corrected learning-rate schedule and held-out early stopping) produced:

	AK-Freq	AK-NEF	AK-NEI
released model (100k pairs)	81.83%	31.21%	47.79%
train-mid (500k pairs)	81.59%	30.97%	48.30%

Five times the ranking supervision changed nothing measurable. Held-out dev loss during that run never once beat having no sparse table at all:

after	dev loss	dev top-1
no table (start)	1.8582	54.18%
batch 1	1.8785	47.23%
batch 2	1.8719	49.07%
batch 3	1.9192	47.82%
batch 4	1.9208	48.08%
batch 5	1.9182	48.67%

Per-batch training loss on *fresh* data climbed ($1.58 \rightarrow 1.90 \rightarrow 2.46$), which is the generalisation gap stated directly. The learning rate annealed exactly as designed (0.05×0.631^n : 0.0500, 0.0315, 0.0199, 0.0126, 0.0079), so this is not a schedule problem.

And the γ sweep — the stated falsification test — came back unmoved:

γ	released model	train-mid
0.0 (heuristic only)	81.02%	80.83%
0.3 (shipped)	81.83%	81.59%
0.5	—	81.45%
0.7	—	79.46%
1.0 (learned only)	76.47%	74.05%

γ still peaks at 0.3, and the standalone learned model got *worse* with more data ($76.47\% \rightarrow 74.05\%$). **The reranker’s problem is not data volume.**

18.2 The likely cause, and the corrected next step

`W_DENSE` — the 29 dense weights — **has never been retrained by this pipeline.** `train.rs` imports it as a frozen constant and fits only the sparse table; the generator named in `reranker_weights.rs` does not exist in the repository. The `v5` container now carries fresh μ and σ so the features are standardised against the current model, but the *weights applied to them* still come from some earlier, lost training run.

So what the system calls “the learned model” is stale dense weights plus a freshly-trained sparse table, and it is unsurprising that the combination loses to a three-parameter heuristic. Adding sparse capacity on top of misfitted dense weights cannot fix that, which is what the two experiments above measured.

The corrected step is to **fit the dense weights and the sparse table jointly** against the current EM/LM output, under the same softmax objective, with L2 regularisation and the held-out early stopping that already exists. Only after that is a full 3.59M run worth its four hours.

18.3 What would falsify the remaining plan

- If a re-measured oracle@50 falls materially below 94.3%, the ranking/generation split is wrong and generation work should take priority.
- If jointly refitting the dense weights still leaves γ peaked at 0.3, the feature set — not the fitting — is the limit, and the discriminative stage should be replaced rather than repaired.

Chapter 19

References

- Chen, S. F. and Goodman, J. (1999). *An Empirical Study of Smoothing Techniques for Language Modeling*. Computer Speech & Language 13(4), 359–394.
- Kneser, R. and Ney, H. (1995). *Improved Backing-off for M-gram Language Modeling*. ICASSP.
- Li, H., Zhang, M. and Su, J. (2004). *A Joint Source-Channel Model for Machine Transliteration*. ACL.
- Madhani, Y. et al. (2023). *Aksharantar: Open Indic-language Transliteration Datasets and Models for the Next Billion Users*. Findings of EMNLP. <https://aclanthology.org/2023.findings-emnlp.4/>
- Rabiner, L. R. (1989). *A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition*. Proceedings of the IEEE 77(2), 257–286. (Scaling of the forward-backward recursions, §5.3.)
- Garbe, W. *SymSpell: 1000x faster spelling correction*. <https://github.com/wolfgarbe/SymSpell>

Chapter 20

Glossary

Akshara — the orthographic syllable of Brahmic scripts; the unit this system models.

Chunk — a contiguous run of 0–5 Roman characters emitted by one akshara.

Continuation probability — in Kneser-Ney, the probability of a token based on how many *distinct contexts* it appears in, not how often it appears.

Halanta / virama (◌्) — the vowel-killer diacritic; glues consonants into conjuncts.

Matra — a dependent vowel sign attached to a consonant. Half of this system’s native errors are matra-only.

Oracle@k — the fraction of cases whose gold answer appears anywhere in the top k candidates; the ceiling a perfect reranker could reach.

Purnabiram (।) — the Devanagari full stop.

Schwa — the inherent vowel of a bare Devanagari consonant, written or omitted inconsistently in Roman input.

Tropical semiring — $(\min, +)$ arithmetic on negative log probabilities; turns maximum-probability search into shortest-path search.